Homework 1

Question 2.1

Q: Describe a situation or problem from your job, everyday life, current events, etc., for which a classification model would be appropriate. List some (up to 5) predictors that you might use.

A: In my job, we have customers making purchases with us on the app and one problem that we can tackle is to identify which customers are likely to be at risk of churn. A classification model will be appropriate to solve this problem as we will be able to identify customers who are likely to churn.

Some predictors we can use are:

- Average order frequency
- Average order value
- Days since last order
- Whether the customer is a new customer
- City of the customer

Question 2.2.1

Q: Using the support vector machine function ksvm contained in the R package kernlab, find a good classifier for this data. Show the equation of your classifier, and how well it classifies the data points in the full data set.

A: The following code takes the approach below to find a good classifier for this data:

- Install and import the "kernlab" library
- Read the "credit_card_data-headers.txt" data into a dataframe
- To find the best value of C (cost parameter) for the model
    - Set a seed number to ensure reproducible results
    - Loop through a set of values of C
    - Train the model using the first 10 columns as predictors and the 11th column as the response, with the "vanilladot" kernel, predictor scaling set to TRUE, and the chosen value of C for the loop
    - Get the predictions of each data point from the model and compare with the actual response values to calculate the model accuracy
    - Compare the model accuracies of each model and return only the best performing value of C and its corresponding model accuracy
- Using the best model, extract the y-intercept and coefficients of predictors to derive the equation of the classifier

With this approach, the best value of C is `0.005` , and the equation (rounded to 3 significant figures) of the classifier is:

`R1 ~ 0.0611 - 0.004(A1) - 0.00293(A2) + 0.00408(A3) + 0.051(A8) + 0.889(A9) - 0.0647(A10) + 0.0526(A11) + 0.00174(A12) - 0.0148(A14) + 0.107(A15)`

The accuracy of the model is: `86.4%`

However, since the model is trained on the entire dataset, it does not necessarily mean that we already have a good model. This model may not perform well against new data and we may have an overfitted model such that it achieves a high accuracy rate on trained data but not on new, unseen data.

```
# Installing & importing packages
library(kernlab)
```

```
# Read credit card data into df
directory <- "/path/to/file/"
df <- read.table(paste0(directory, "credit_card_data-headers.txt"), header = T, sep = "\t")
```

```
# Set seed for reproducibility
set.seed(31)
```

```
# Find the best performing value of C based on the accuracy of predictions
C_list <- list("0.001", "0.005", "0.01", "0.05", "0.1", "0.5", "1", "5", "10", "50", "100")
best_i <- 0
best_accuracy <- 0

for (i in C_list) {
  print(i)
  model <- ksvm(as.matrix(df[, 1:10]), as.factor(df[, 11]), type="C-svc", kernel="vanilladot", C=i, scaled=TRUE)
  pred <- predict(model, df[, 1:10])
  accuracy <- sum(pred == df[, 11]) / nrow(df)
  if (accuracy > best_accuracy) {
    best_i <- i
    best_accuracy <- accuracy
  }
}

cat("The best value for C is: ", best_i)
```

Output:

The best value for C is: `0.005`

```
# Train the model using the best value of C
final_model <- ksvm(as.matrix(df[, 1:10]), as.factor(df[, 11]), type="C-svc", kernel="vanilladot", C=best_i, scaled=TRUE)
final_pred <- predict(final_model, df[, 1:10])
final_accuracy <- sum(final_pred == df[, 11]) / nrow(df)

cat("Using the best value for C, the accuracy is: ", final_accuracy*100, "%")
```

Output:

Using the best value for C, the accuracy is: `86.39144 %`

```
# Calculate coefficients of a1…am
a <- colSums(final_model@xmatrix[[1]] * final_model@coef[[1]])

# Calculate a0
a0 <- -final_model@b
```

```
# Show the equation of the classifier and accuracy of the model
equation <- a0

for (name in names(a)) {
  if (substring(a[name], 1, 1) == "-") {
    equation <- paste0(equation, " - ", substring(a[name], 2), "*", name)
  } else {
    equation <- paste0(equation, " + ", a[name], "*", name)
  }
}

cat("The equation of the classifier is:", equation, "\n")
cat("The accuracy of the classifier is:", final_accuracy*100, "%")
```

Output:

The equation of the classifier is:
`0.0611144124422101 - 0.00399815004782269*A1 - 0.00292870529606904*A2 + 0.00408042087313143*A3 + 0.0510361524399184*A8 + 0.889321257035186*A9 - 0.(`

The accuracy of the classifier is: `86.39144 %`

Question 2.2.2

Q: You are welcome, but not required, to try other (nonlinear) kernels as well; we're not covering them in this course, but they can sometimes be useful and might provide better predictions than vanilladot.

A: Using a similar approach as in Q2.2.1, we loop through a set of values of C as well to identify the best performing value of C for each type of non-linear kernel available in the ksvm model.

With this, we have a nested loop, looping through each type of kernel, and for each kernel, we loop through the set of values of C to find out the best model performance for each kernel.

The performances for each kernel are as follows:

- Radial Basis kernel: (C value: 100, accuracy: 95.1%)
- Polynomial kernel: (C value: 0.005, accuracy: 86.4%)
- Hyperbolic Tangent kernel: (C value: 0.01, accuracy: 86.2%)
- Laplacian kernel: (C value: 100, accuracy: 100%)
- Bessel kernel: (C value: 100, accuracy: 92.5%)

- ANOVA Radial Basis kernel: (C value: 100, accuracy: 90.7%)
- Spline kernel: (C value: 10, accuracy: 97.9%)

Similarly, since the models are trained with the full dataset and not validated with new data, the higher accuracy of the models could be due to a greater degree of overfitting, especially for the Laplacian kernel with a 100% accuracy rate

```
# Set seed for reproducibility
set.seed(31)
```

```
# Training classifier with a range of kernels and values of C
kernel_list <- list("rbfdot", "polydot", "tanhdot", "laplacedot", "besseldot", "anovadot", "splinedot")
C_list <- list("0.001", "0.005", "0.01", "0.05", "0.1", "0.5", "1", "5", "10", "50", "100")

for (k in kernel_list) {
  best_i <- 0
  best_accuracy <- 0

  ign <- capture.output({
    for (i in C_list) {
      model <- ksvm(as.matrix(df[, 1:10]), as.factor(df[, 11]), type="C-svc", kernel=k, C=i, scaled=TRUE)
      pred <- predict(model, df[, 1:10])
      accuracy <- sum(pred == df[, 11]) / nrow(df)
      if (accuracy > best_accuracy) {
        best_i <- i
        best_accuracy <- accuracy
      }
    }
  })

  cat("The best value of C for", k, "kernel is:", best_i, "\n")
  cat("Using the best value of C, the accuracy for", k, "is:", best_accuracy*100, "%", "\n\n")
}
```

Output:

The best value of C for rbfdot kernel is: `100` . Using the best value of C, the accuracy for rbfdot is: `95.10703 %`

The best value of C for polydot kernel is: `0.005` . Using the best value of C, the accuracy for polydot is: `86.39144 %`

The best value of C for tanhdot kernel is: `0.01` . Using the best value of C, the accuracy for tanhdot is: `86.23853 %`

The best value of C for laplacedot kernel is: `100` . Using the best value of C, the accuracy for laplacedot is: `100 %`

The best value of C for besseldot kernel is: `100` . Using the best value of C, the accuracy for besseldot is: `92.50765 %`

The best value of C for anovadot kernel is: `100` . Using the best value of C, the accuracy for anovadot is: `90.67278 %`

The best value of C for splinedot kernel is: `10` . Using the best value of C, the accuracy for splinedot is: `97.85933 %`

Question 2.2.3

Q: Using the k-nearest-neighbors classification function kknn contained in the R kknn package, suggest a good value of k, and show how well it classifies that data points in the full data set. Don't forget to scale the data (scale=TRUE in kknn).

A: The following code takes the approach below to find a good value of k for the k-nearest-neighbors classifier for this data, as well as the performance of this model:

- Install and import the "kknn" & "caret" library
- Prepare train and test subsets from the credit card dataframe
  - Set a seed number to ensure reproducibility
  - Find the number of data points in the credit card dataframe
  - Sample about 80% of the dataframe to be in the training dataset, and the remaining 20% to be in the test dataset
- To find the best value of k for the model:
  - Loop through a set of values of k (from 1 to 100 in this case)
  - Train the model using the training dataset with the first 10 columns as predictors and the 11th column as the response, with the "optimal" kernel, predictor scaling set to TRUE, distance parameter to be 1, and the chosen value of k for the loop
  - Fit the model and use it to predict on the test dataset to compare with the actual response values to calculate the model accuracy
  - Compare the model accuracies for each value of k and return only the best performing value of k and its corresponding model accuracy
  - Plot the confusion matrix to observe more in-depth metrics of the predictions

With this approach, the best value of k is `44` and the model accuracy with this k is about `86.3%`

Confusion Matrix

Prediction 0 - Actual 0 (True Negative) : 67

Prediction 1 - Actual 0 (False Positive): 3

Prediction 0 - Actual 1 (False Negative): 15

Prediction 1 - Actual 1 (True Positive) : 46

Sensitivity : `95.71%`
Specificity : `75.41%`

```
# Installing & importing packages
library(kknn)
library(caret)
```

```
# Set seed for reproducibility
set.seed(31)
```

```
# Create train-test split
df_len <- dim(df)[1]
test_sample <- sample(1:df_len, size=round(0.2*df_len), replace=FALSE)
train_set <- df[-test_sample,]
test_set <- df[test_sample,]
```

```
# Train the KNN classifier and find the best value of k
best_i <- 0
best_accuracy <- 0

for (i in 1:100) {
  kknn_model <- kknn(as.factor(R1)~., train_set, test_set, k=i, distance=1, kernel="optimal", scale=T)
  fit <- fitted(kknn_model)
  accuracy <- sum(test_set$R1 == fit) / nrow(test_set)
  if (accuracy > best_accuracy) {
    best_i <- i
    best_accuracy <- accuracy
  }
}

cat("Best value of k is:", best_i, "\n")
cat("Using the best value of k, the accuracy is:", best_accuracy*100, "%")
```

Output:

Best value of k is: `44`

Using the best value of k, the accuracy is: `86.25954 %`

```
confusionMatrix(data=fit, reference=as.factor(test_set$R1))
```